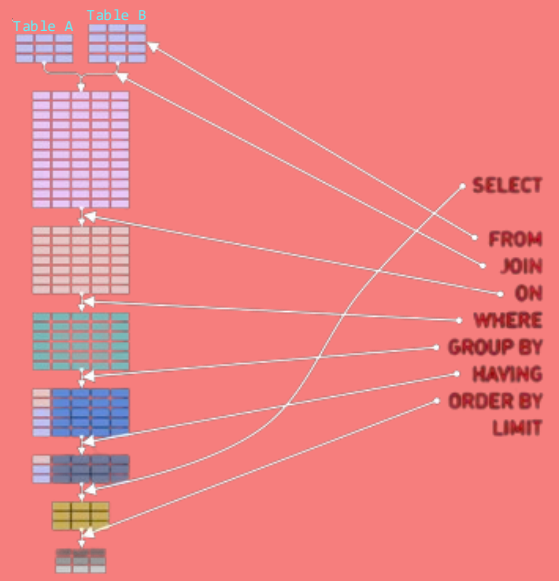
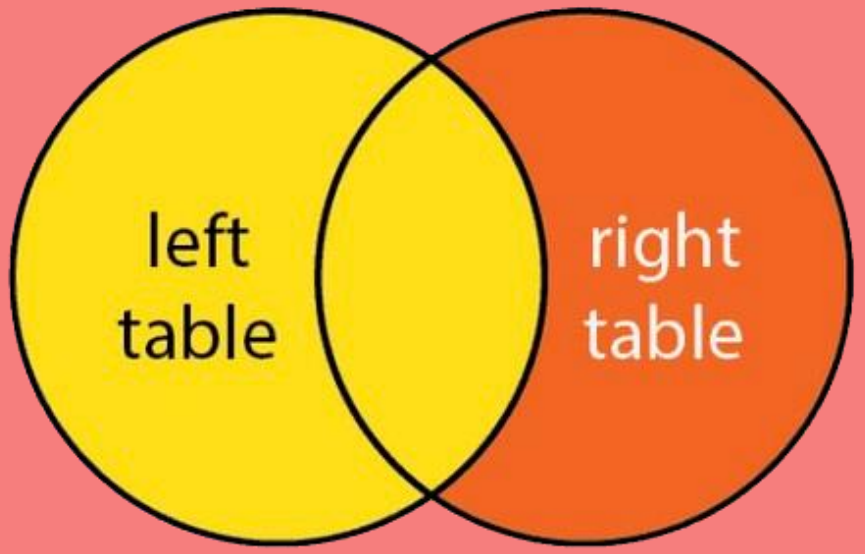
SQL QUERY – Visualização

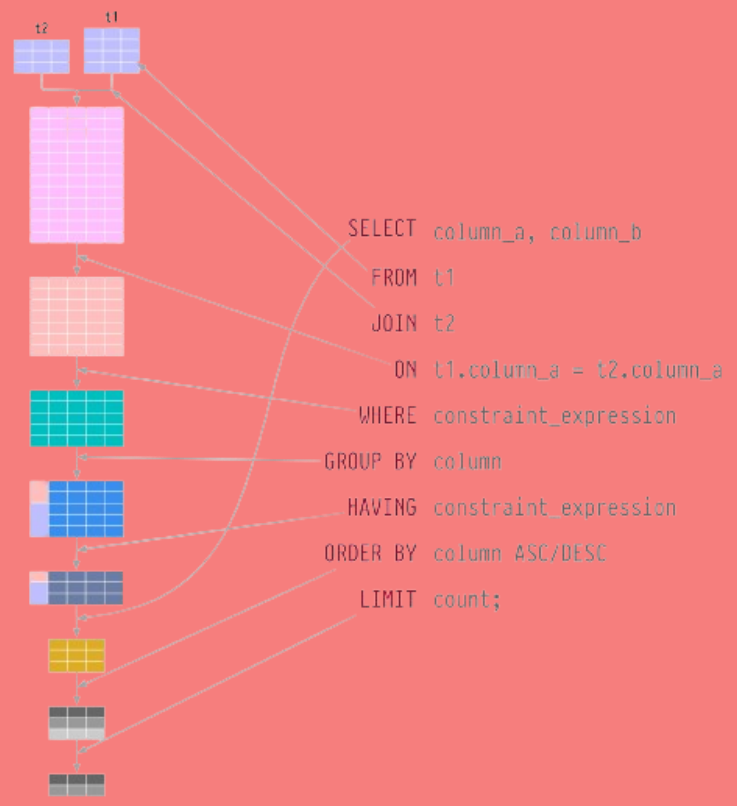




LEFT JOIN = JOIN

SQL QUERY - Visualização

SQL QUERY - Ordem de execução




```
SELECT
        clientes.id as cliente_id,
        nome,
        entregas.id as entregas_id
FROM clientes
LEFT JOIN entregas
ON clientes.id = entregas.cliente_id
GROUP BY clientes.id;
```

SQL QUERY - Visualização

SQL QUERY – INNER JOIN = JOIN

Um cliente requisita várias entregas

clientes

PK

id	nome	nif
1	Paulo	111
2	Rui	222
3	Bruno	333
4	Áureo	444
5	Leandro	555
6	Bruno	666

entregas

PK FK

id	cliente_id	preco	data_entrega
1	1	400.00	13/12/2023 13:48
2	4	15.00	NULL
3	6	30.00	NULL
4	2	1300.00	NULL
5	1	600.00	13/12/2023 13:48
6	4	500.00	NULL
7	6	33.00	NULL

PK – Primary Key

É uma coluna ou por vezes várias, que identifica exclusivamente uma linha numa tabela

FK – Foreign Key

Estabelece uma relação com uma PK de outra tabela através de valores partilhados

SQL QUERY - Visualização

SQL QUERY – INNER JOIN = JOIN

clientes

PK

id	nome	nif
1	Paulo	111
2	Rui	222
3	Bruno	333
4	Áureo	444
5	Leandro	555
6	Bruno	666

entregas

PK FK

id	cliente_id	preco	data_entrega
1	1	400.00	13/12/2023 13:48
2	4	15.00	NULL
3	6	30.00	NULL
4	2	1300.00	NULL
5	1	600.00	13/12/2023 13:48
6	4	500.00	NULL
7	6	33.00	NULL

Exemplo real

Duas tabelas, clientes e entregas, retiradas de um CRM como por exemplo o SAP.

O Departmento de Vendas necessita de saber quantas entregas tem cada clientes, mesmo aqueles que ainda não solicitaram qualquer entrega.

SQL QUERY - Visualização

SQL QUERY – INNER JOIN = JOIN

PK

id	nome	nif
1	Paulo	111
2	Rui	222
3	Bruno	333
4	Áureo	444
5	Leandro	555
6	Bruno	666

entregas

PK FK

id	cliente_id	preco	data_entrega
1	1	400.00	13/12/2023 13:48
2	4	15.00	NULL
3	6	30.00	NULL
4	2	1300.00	NULL
5	1	600.00	13/12/2023 13:48
6	4	500.00	NULL
7	6	33.00	NULL

client_id	nome	entregas_id
1	Paulo	2

```
SELECT
        clientes.id as cliente_id,
        nome,
        entregas.id as entregas_id
FROM clientes
LEFT JOIN entregas
ON clientes.id = entregas.cliente_id
GROUP BY clientes.id;
```

1. É criada uma nova tabela com os atributos das duas tabelas do LEFT JOIN
2. É percorrida a coluna id da tabela clientes e identificação da sua correspondência na tabela entregas. Caso não seja encontrado nenhuma correspondência os atributos da tabela entregas são colocados a null

ON clientes.id = entregas.client_id

1 1

SQL QUERY - Visualização

SQL QUERY – INNER JOIN = JOIN

PK

id	nome	nif
1	Paulo	111
2	Rui	222
3	Bruno	333
4	Áureo	444
5	Leandro	555
6	Bruno	666

entregas

PK FK

id	cliente_id	preco	data_entrega
1	1	400.00	13/12/2023 13:48
2	4	15.00	NULL
3	6	30.00	NULL
4	2	1300.00	NULL
5	1	600.00	13/12/2023 13:48
6	4	500.00	NULL
7	6	33.00	NULL

client_id	nome	entregas_id
1	Paulo	2
2	Rui	4

```
SELECT
        clientes.id as cliente_id,
        nome,
        entregas.id as entregas_id
FROM clientes
LEFT JOIN entregas
ON clientes.id = entregas.cliente_id
GROUP BY clientes.id;
```

ON clientes.id = entregas.client_id

2 2

SQL QUERY - Visualização

SQL QUERY – INNER JOIN = JOIN

PK

id	nome	nif
1	Paulo	111
2	Rui	222
3	Bruno	333
4	Áureo	444
5	Leandro	555
6	Bruno	666

entregas

PK FK

id	cliente_id	preco	data_entrega
1	1	400.00	13/12/2023 13:48
2	4	15.00	NULL
3	6	30.00	NULL
4	2	1300.00	NULL
5	1	600.00	13/12/2023 13:48
6	4	500.00	NULL
7	6	33.00	NULL

client_id	nome	entregas_id
1	Paulo	2
2	Rui	1
3	Bruno	null

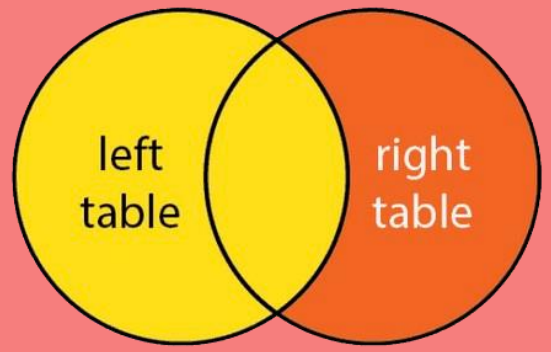
```
SELECT
        clientes.id as cliente_id,
        nome,
        entregas.id as entregas_id
FROM clientes
LEFT JOIN entregas
ON clientes.id = entregas.cliente_id
GROUP BY clientes.id;
```

Não existe correspondência

ON clientes.id = entregas.client_id

3 ≠ 3

LEFT JOIN (Conclusão)



O LEFT JOIN serve para obter todos os registos da tabela da esquerda, mesmo quando não existe correspondência na tabela da direita.

Em termos simples: O LEFT JOIN mostra tudo o que está na tabela da esquerda e, se houver relação com a tabela da direita, mostra esses dados; caso contrário, aparecem valores NULL.

O LEFT JOIN é usado quando se pretende:

- Manter todos os registos da tabela principal (esquerda)
- Identificar dados em falta na tabela relacionada
- Garantir resultados completos, mesmo sem correspondência
- Analisar relações opcionais, não obrigatórias